# Identificação do projeto

**Título do projeto:** Fatores operacionais, atrasos de entrega e satisfação do cliente - Brazilian E-Commerce (Olist)
**Integrantes do grupo:** Caetano Padoin, Davi Henrique e Matheus Brehm
**Turma:** Sexto A
**Descrição breve do problema ou contexto:** A Olist é um marketplace brasileiro. Com ~99 mil pedidos reais e anonimizados (2016-2018), investigamos quais fatores operacionais (tempo de entrega, frete, região, categoria, pagamento) estão associados à satisfação do cliente e a atrasos logísticos.

Preencha este notebook ao longo dos dois RAs. Registre decisões e interpretações em poucas frases e acrescente as células de código necessárias em cada etapa.

---

# RA1 - Data Ingestion, Cleaning and Initial Analysis

Nesta etapa, carregue, prepare e explore os dados do projeto.

## 1. Contexto e objetivo

Apresente brevemente o problema investigado e o que o grupo pretende descobrir com os dados.

**Resposta do grupo:**

A Olist é um marketplace brasileiro que conecta vendedores a grandes marketplaces; o dataset público reúne ~99 mil pedidos reais e anonimizados (2016-2018), cobrindo compra, pagamento, entrega e avaliação.

**Problema:** entender quais fatores operacionais - tempo de entrega, frete, região, categoria do produto e forma de pagamento - estão associados à satisfação do cliente (nota de avaliação) e a atrasos logísticos.

**Perguntas analíticas** (serão respondidas nas próximas etapas; detalhes em `docs/01_problema_e_perguntas.md`):

1. Qual a relação entre o atraso na entrega (data real vs. estimada) e a nota de avaliação?
2. Quais estados/regiões apresentam maior incidência de atrasos?
3. Existe relação entre a categoria do produto e a taxa de atraso ou a nota de avaliação?
4. Como o valor do frete se relaciona com a distância vendedor-cliente e com a satisfação?

5. Qual a distribuição de métodos e parcelas de pagamento, e como se relaciona com o valor do pedido?

## 2. Fonte e ingestão dos dados

- Identifique a fonte dos dados, incluindo link ou referência de acesso.
- Descreva brevemente o dataset e o que seus registros representam.
- Explique como os dados foram carregados e implemente a ingestão.

**Resposta do grupo:**

**Fonte:** *Brazilian E-Commerce Public Dataset by Olist* - https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce (CC BY-NC-SA 4.0, uso educacional). Cada registro representa um pedido, cliente, item, pagamento, avaliação, produto, vendedor ou ponto de geolocalização, conforme o arquivo.

**Formato e volumetria:** 9 arquivos CSV ligados por `order_id`, `customer_id`, `product_id` e `seller_id` (~99 mil pedidos e ~1 milhão de pontos de geolocalização; contagem por arquivo no resultado da ingestão abaixo e em `docs/02_fontes_de_dados.md`).

**Ingestão:** por código (célula abaixo): baixa os 9 CSVs originais de um mirror público no GitHub (mesmos arquivos do Kaggle, sem exigir credencial de API) e salva em `data/raw/`, que nunca é editado manualmente.

# 01 - Ingestão dos Dados

**Fonte**: Brazilian E-Commerce Public Dataset by Olist - https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce (licença CC BY-NC-SA 4.0, uso educacional).

Para que a ingestão seja **reproduzível por código sem exigir credenciais de API do Kaggle**, este notebook baixa os mesmos 9 arquivos CSV originais a partir de um mirror público no GitHub (`ckoliveiraa/pipeline-olist`), que replica os arquivos do dataset oficial com os mesmos nomes e conteúdo (comprimidos em `.gz`). Os arquivos são descompactados e salvos em `data/raw/`, que nunca é editado manualmente.

Ver `docs/02_fontes_de_dados.md` para detalhes de cada arquivo.

In [1]:
```
import gzip
import shutil
import urllib.request
from pathlib import Path

import pandas as pd

RAW_DIR = Path("../data/raw")
RAW_DIR.mkdir(parents=True, exist_ok=True)

BASE_URL = "https://raw.githubusercontent.com/ckoliveiraa/pipeline-olist/main/raw"
```

```
ARQUIVOS = [
    "olist_customers_dataset",
    "olist_geolocation_dataset",
    "olist_order_items_dataset",
    "olist_order_payments_dataset",
    "olist_order_reviews_dataset",
    "olist_orders_dataset",
    "olist_products_dataset",
    "olist_sellers_dataset",
    "product_category_name_translation",
]
```

In [2]:
```
def baixar_e_descomprimir(nome: str) -> Path:
    """Baixa <nome>.csv.gz do mirror e salva <nome>.csv em data/raw, se ainda não existir."""
    destino_csv = RAW_DIR / f"{nome}.csv"
    if destino_csv.exists():
        return destino_csv

    url = f"{BASE_URL}/{nome}.csv.gz"
    destino_gz = RAW_DIR / f"{nome}.csv.gz"
    urllib.request.urlretrieve(url, destino_gz)

    with gzip.open(destino_gz, "rb") as f_in, open(destino_csv, "wb") as f_out:
        shutil.copyfileobj(f_in, f_out)
    destino_gz.unlink()
    return destino_csv


caminhos = {nome: baixar_e_descomprimir(nome) for nome in ARQUIVOS}
caminhos
```

Out[2]:
```
{'olist_customers_dataset':
WindowsPath('../data/raw/olist_customers_dataset.csv'),
 'olist_geolocation_dataset':
WindowsPath('../data/raw/olist_geolocation_dataset.csv'),
 'olist_order_items_dataset':
WindowsPath('../data/raw/olist_order_items_dataset.csv'),
 'olist_order_payments_dataset':
WindowsPath('../data/raw/olist_order_payments_dataset.csv'),
 'olist_order_reviews_dataset':
WindowsPath('../data/raw/olist_order_reviews_dataset.csv'),
 'olist_orders_dataset': WindowsPath('../data/raw/olist_orders_dataset.csv'),
 'olist_products_dataset': WindowsPath('../data/raw/olist_products_dataset.csv'),
 'olist_sellers_dataset': WindowsPath('../data/raw/olist_sellers_dataset.csv'),
 'product_category_name_translation':
WindowsPath('../data/raw/product_category_name_translation.csv')}
```

# Carregamento e verificação da ingestão

In [3]:
```
dados = {nome: pd.read_csv(caminho) for nome, caminho in caminhos.items()}

resumo = pd.DataFrame(
    [(nome, df.shape[0], df.shape[1]) for nome, df in dados.items()],
    columns=["arquivo", "linhas", "colunas"],
)
resumo
```

Out[3]:

| | arquivo | linhas | colunas |
|---|---|---|---|
| 0 | olist_customers_dataset | 99441 | 5 |
| 1 | olist_geolocation_dataset | 1000163 | 5 |
| 2 | olist_order_items_dataset | 112650 | 7 |
| 3 | olist_order_payments_dataset | 103886 | 5 |
| 4 | olist_order_reviews_dataset | 99224 | 7 |
| 5 | olist_orders_dataset | 99441 | 8 |
| 6 | olist_products_dataset | 32951 | 9 |
| 7 | olist_sellers_dataset | 3095 | 4 |
| 8 | product_category_name_translation | 71 | 2 |

A ingestão carregou os 9 arquivos com sucesso. As contagens de linhas conferem com as estatísticas públicas conhecidas do dataset (ex.: ~99.441 pedidos, ~1.000.163 registros de geolocalização), confirmando a integridade do download.

Próxima etapa: `02_avaliacao_qualidade.ipynb`, que carrega os mesmos arquivos de `data/raw/` para investigar problemas de qualidade.

## 3. Exploração inicial dos dados

Investigue a quantidade de registros e colunas, os tipos de dados, os valores ausentes e outras características relevantes da base.

# 02 - Compreensão e Avaliação da Qualidade dos Dados

Este notebook carrega os 9 arquivos de `data/raw/` (gerados em `01_ingestao.ipynb`) e investiga, para cada um, os problemas de qualidade relevantes: valores ausentes, registros duplicados, tipos de dados incorretos, formatos inconsistentes, valores inválidos, categorias inconsistentes e outliers.

Os achados aqui orientam diretamente as decisões de limpeza em `03_limpeza_preparacao.ipynb`.

In [1]:
```
import pandas as pd

pd.set_option("display.max_columns", None)

RAW = "../data/raw/"

orders = pd.read_csv(RAW + "olist_orders_dataset.csv")
customers = pd.read_csv(RAW + "olist_customers_dataset.csv")
items = pd.read_csv(RAW + "olist_order_items_dataset.csv")
payments = pd.read_csv(RAW + "olist_order_payments_dataset.csv")
reviews = pd.read_csv(RAW + "olist_order_reviews_dataset.csv")
products = pd.read_csv(RAW + "olist_products_dataset.csv")
sellers = pd.read_csv(RAW + "olist_sellers_dataset.csv")
geo = pd.read_csv(RAW + "olist_geolocation_dataset.csv")
cat_translation = pd.read_csv(RAW + "product_category_name_translation.csv")

tabelas = {
    "orders": orders, "customers": customers, "items": items, "payments":
payments,
    "reviews": reviews, "products": products, "sellers": sellers, "geo": geo,
```

```
    "cat_translation": cat_translation,
}
```

# Visão geral: formato, tipos e valores ausentes por tabela

In [2]:

```
for nome, df in tabelas.items():
    print(f"=== {nome} - {df.shape[0]} linhas x {df.shape[1]} colunas ===")
    print(df.dtypes.to_string())
    nulos = df.isnull().sum()
    nulos = nulos[nulos > 0]
    if len(nulos):
        print("\nValores ausentes:")
        print(nulos.to_string())
    print()
```

```
=== orders - 99441 linhas x 8 colunas ===
order_id                          str
customer_id                       str
order_status                      str
order_purchase_timestamp          str
order_approved_at                 str
order_delivered_carrier_date      str
order_delivered_customer_date     str
order_estimated_delivery_date     str

Valores ausentes:
order_approved_at                 160
order_delivered_carrier_date     1783
order_delivered_customer_date    2965

=== customers - 99441 linhas x 5 colunas ===
customer_id                     str
customer_unique_id              str
customer_zip_code_prefix      int64
customer_city                   str
customer_state                  str

=== items - 112650 linhas x 7 colunas ===
order_id                      str
order_item_id               int64
product_id                    str
seller_id                     str
shipping_limit_date           str
price                     float64
freight_value             float64

=== payments - 103886 linhas x 5 colunas ===
order_id                        str
payment_sequential          int64
payment_type                  str
payment_installments        int64
payment_value             float64

=== reviews - 99224 linhas x 7 colunas ===
review_id                       str
order_id                        str
review_score                  int64
review_comment_title            str
review_comment_message          str
review_creation_date            str
review_answer_timestamp         str

Valores ausentes:
review_comment_title      87656
review_comment_message    58247

=== products - 32951 linhas x 9 colunas ===
product_id                          str
product_category_name               str
product_name_lenght             float64
product_description_lenght      float64
product_photos_qty              float64
product_weight_g                float64
product_length_cm               float64
product_height_cm               float64
product_width_cm                float64
```

```
Valores ausentes:
product_category_name         610
product_name_lenght           610
product_description_lenght    610
product_photos_qty            610
product_weight_g                2
product_length_cm               2
product_height_cm               2
product_width_cm                2

=== sellers - 3095 linhas x 4 colunas ===
seller_id                   str
seller_zip_code_prefix    int64
seller_city                 str
seller_state                str

=== geo - 1000163 linhas x 5 colunas ===
geolocation_zip_code_prefix      int64
geolocation_lat                float64
geolocation_lng                float64
geolocation_city                   str
geolocation_state                  str

=== cat_translation - 71 linhas x 2 colunas ===
product_category_name            str
product_category_name_english    str
```

**Achado 1 - tipos de dados incorretos:** todas as colunas de data/hora ( `orders` , `items` , `reviews` ) estão como texto ( `object` ), não `datetime` . Isso impede cálculos de prazo de entrega e precisa ser corrigido na limpeza.

**Achado 2 - valores ausentes legítimos:**

- `orders` : `order_approved_at` (160), `order_delivered_carrier_date` (1.783) e `order_delivered_customer_date` (2.965) ficam nulos quando o pedido nunca chegou a essa etapa (cancelado, indisponível, ainda em processamento). Verificamos isso abaixo cruzando com `order_status` .
- `reviews` : `review_comment_title` e `review_comment_message` têm milhares de nulos porque são campos de texto opcionais - o cliente pode avaliar sem escrever comentário.
- `products` : ~610 produtos sem `product_category_name` e metadados associados, e 2 produtos sem dimensões físicas. São produtos cadastrados de forma incompleta pelos vendedores.

In [3]:
```
# Confirma que os nulos de data em 'orders' se concentram em pedidos não concluídos
orders.groupby("order_status")[["order_approved_at",
"order_delivered_carrier_date", "order_delivered_customer_date"]].apply(
    lambda g: g.isnull().sum()
)
```

Out[3]:

| order_status | order_approved_at | order_delivered_carrier_date | order_delivered_customer_date |
|---|---|---|---|
| approved | 0 | 2 | 2 |
| canceled | 141 | 550 | 619 |
| created | 5 | 5 | 5 |
| delivered | 14 | 2 | 8 |
| invoiced | 0 | 314 | 314 |
| processing | 0 | 301 | 301 |
| shipped | 0 | 0 | 1107 |
| unavailable | 0 | 609 | 609 |

# Registros duplicados

In [4]:
```
for nome, df in tabelas.items():
    print(nome, "- duplicatas de linha inteira:", df.duplicated().sum())
```

```
orders - duplicatas de linha inteira: 0
customers - duplicatas de linha inteira: 0
items - duplicatas de linha inteira: 0
payments - duplicatas de linha inteira: 0
reviews - duplicatas de linha inteira: 0
products - duplicatas de linha inteira: 0
sellers - duplicatas de linha inteira: 0
geo - duplicatas de linha inteira: 261831
cat_translation - duplicatas de linha inteira: 0
```

In [5]:
```
# geo concentra quase todas as duplicatas do dataset: investigar
dup_geo = geo.duplicated().sum()
print(f"geolocation: {dup_geo} linhas duplicadas de {len(geo)}
({dup_geo/len(geo):.1%})")
geo[geo.duplicated(keep=False)].sort_values("geolocation_zip_code_prefix").head(6)
```

```
geolocation: 261831 linhas duplicadas de 1000163 (26.2%)
```

Out[5]:

| | geolocation_zip_code_prefix | geolocation_lat | geolocation_lng | geolocation_city | geolocation_state |
|---|---|---|---|---|---|
| 851 | 1001 | -23.549825 | -46.633970 | sao paulo | SP |
| 1062 | 1001 | -23.550498 | -46.634338 | sao paulo | SP |
| 1246 | 1001 | -23.549292 | -46.633559 | sao paulo | SP |
| 897 | 1001 | -23.549292 | -46.633559 | sao paulo | SP |
| 99 | 1001 | -23.549292 | -46.633559 | sao paulo | SP |
| 864 | 1001 | -23.549825 | -46.633970 | sao paulo | SP |

**Achado 3 - duplicatas relevantes:** `olist_geolocation_dataset.csv` tem ~26% de linhas 100% duplicadas (mesmo CEP, mesma lat/lng, mesma cidade/estado). Isso infla o arquivo sem adicionar nenhuma informação nova e deve ser removido na limpeza.

As demais tabelas não têm duplicatas de linha inteira.

# Chaves que deveriam ser únicas

In [6]:
```
print("order_id duplicado em orders:", orders["order_id"].duplicated().sum())
print("review_id duplicado em reviews:", reviews["review_id"].duplicated().sum())
print("review_id + order_id duplicado:", reviews.duplicated(subset=["review_id",
"order_id"]).sum())

exemplo =
reviews[reviews["review_id"].duplicated(keep=False)].sort_values("review_id")
exemplo[["review_id", "order_id", "review_score"]].head(4)
```

```
order_id duplicado em orders: 0
review_id duplicado em reviews: 814
review_id + order_id duplicado: 0
```

Out[6]:

| | review_id | order_id | review_score |
|---|---|---|---|
| **46678** | 00130cbe1f9d422698c812ed8ded1919 | dfcdfc43867d1c1381bfaf62d6b9c195 | 1 |
| **29841** | 00130cbe1f9d422698c812ed8ded1919 | 04a28263e085d399c97ae49e0b477efa | 1 |
| **90677** | 0115633a9c298b6a98bcbe4eee75345f | 78a4201f58af3463bdab842eea4bc801 | 5 |
| **63193** | 0115633a9c298b6a98bcbe4eee75345f | 0c9850b2c179c1ef60d2855e2751d1fa | 5 |

**Achado 4 - `review_id` não é uma chave única:** 814 valores de `review_id` aparecem em mais de um `order_id` diferente - é uma particularidade conhecida do dataset da Olist (o mesmo texto de avaliação pode ter sido reaproveitado entre pedidos). Como o par `(review_id, order_id)` **é** único, a granularidade correta de análise é o pedido, não a avaliação isolada. Não é um erro a corrigir, é uma característica a documentar.

# Valores inválidos ou impossíveis

In [7]:
```
print("=== items: price e freight_value ===")
print(items[["price", "freight_value"]].describe())
print("price <= 0:", (items["price"] <= 0).sum(), "| freight_value < 0:",
(items["freight_value"] < 0).sum())

print("\n=== payments ===")
print(payments[["payment_value", "payment_installments"]].describe())
print("payment_value <= 0:", (payments["payment_value"] <= 0).sum())
print("payment_installments == 0:", (payments["payment_installments"] == 0).sum())
print(payments["payment_type"].value_counts())
```

```
=== items: price e freight_value ===
               price  freight_value
count  112650.000000  112650.000000
mean      120.653739      19.990320
std       183.633928      15.806405
min         0.850000       0.000000
25%        39.900000      13.080000
50%        74.990000      16.260000
75%       134.900000      21.150000
max      6735.000000     409.680000
price <= 0: 0 | freight_value < 0: 0

=== payments ===
       payment_value  payment_installments
count  103886.000000         103886.000000
mean      154.100380              2.853349
std       217.494064              2.687051
min         0.000000              0.000000
25%        56.790000              1.000000
50%       100.000000              1.000000
75%       171.837500              4.000000
max     13664.080000             24.000000
payment_value <= 0: 9
payment_installments == 0: 2
payment_type
credit_card     76795
boleto          19784
voucher          5775
debit_card       1529
not_defined         3
Name: count, dtype: int64
```

In [8]:
```
payments[payments["payment_value"] <= 0]
```

Out[8]:

| | order_id | payment_sequential | payment_type | payment_installments | payment_value |
|---|---|---|---|---|---|
| **19922** | 8bcbe01d44d147f901cd3192671144db | 4 | voucher | 1 | 0.0 |
| **36822** | fa65dad1b0e818e3ccc5cb0e39231352 | 14 | voucher | 1 | 0.0 |
| **43744** | 6ccb433e00daae1283ccc956189c82ae | 4 | voucher | 1 | 0.0 |
| **51280** | 4637ca194b6387e2d538dc89b124b0ee | 1 | not_defined | 1 | 0.0 |
| **57411** | 00b1cb0320190ca0daa2c88b35206009 | 1 | not_defined | 1 | 0.0 |
| **62674** | 45ed6e85398a87c253db47c2d9f48216 | 3 | voucher | 1 | 0.0 |
| **77885** | fa65dad1b0e818e3ccc5cb0e39231352 | 13 | voucher | 1 | 0.0 |
| **94427** | c8c528189310eaa44a745b8d9d26908b | 1 | not_defined | 1 | 0.0 |
| **100766** | b23878b3e8eb4d25a158f57d96331b18 | 4 | voucher | 1 | 0.0 |

**Achado 5 - pagamentos inválidos:**

- 3 linhas têm `payment_type == "not_defined"` **e** `payment_value == 0`: não representam nenhuma forma de pagamento real, parecem registros de placeholder - candidatas à remoção.
- As outras 6 linhas com `payment_value == 0` são `payment_type == "voucher"`: fazem parte de pagamentos fracionados em múltiplas parcelas de voucher (`payment_sequential` > 1) e são legítimas - não remover.
- 2 linhas têm `payment_installments == 0` com `payment_type == "credit_card"` e valor > 0: fisicamente impossível (mesmo um pagamento à vista conta como 1 parcela neste esquema) - valor inválido a corrigir.

In [9]:
```
print("=== products: dimensões físicas ===")
print(products[["product_weight_g", "product_length_cm", "product_height_cm",
"product_width_cm"]].describe())
print("product_weight_g == 0:", (products["product_weight_g"] == 0).sum())
```

```
=== products: dimensões físicas ===
       product_weight_g  product_length_cm  product_height_cm  \
count      32949.000000       32949.000000       32949.000000
mean        2276.472488          30.815078          16.937661
std         4282.038731          16.914458          13.637554
min            0.000000           7.000000           2.000000
25%          300.000000          18.000000           8.000000
50%          700.000000          25.000000          13.000000
75%         1900.000000          38.000000          21.000000
max        40425.000000         105.000000         105.000000

       product_width_cm
count      32949.000000
mean          23.196728
std           12.079047
min            6.000000
25%           15.000000
50%           20.000000
75%           30.000000
max          118.000000
product_weight_g == 0: 4
```

**Achado 6 - peso zero:** 4 produtos têm `product_weight_g == 0`, o que é fisicamente impossível para um item físico vendido - deve ser tratado como ausente, não como zero real.

# Categorias inconsistentes

In [10]:
```
estados_validos = {
```

```
"AC","AL","AM","AP","BA","CE","DF","ES","GO","MA","MG","MS","MT","PA","PB","PE","PI",
    "PR","RJ","RN","RO","RR","RS","SC","SE","SP","TO",
}
print("estados inválidos em customers:",
(~customers["customer_state"].isin(estados_validos)).sum())
print("estados inválidos em sellers:",
(~sellers["seller_state"].isin(estados_validos)).sum())
print("estados inválidos em geo:",
(~geo["geolocation_state"].isin(estados_validos)).sum())

cats_traduzidas = set(cat_translation["product_category_name"])
cats_produtos = set(products["product_category_name"].dropna())
print("\ncategorias de produto sem tradução em inglês:", cats_produtos -
cats_traduzidas)
```

```
estados inválidos em customers: 0
estados inválidos em sellers: 0
estados inválidos em geo: 0

categorias de produto sem tradução em inglês: {'pc_gamer',
'portateis_cozinha_e_preparadores_de_alimentos'}
```

In [11]:

```
import re

padrao_valido = re.compile(r"^[a-z0-9áéíóúãõâêôçàü\s\-\.\(\)/']*$")

def contar_nomes_invalidos(df, coluna):
    mask = ~df[coluna].fillna("").str.match(padrao_valido)
    return mask

for nome_df, df, coluna in [("customers", customers, "customer_city"), ("sellers",
sellers, "seller_city"), ("geo", geo, "geolocation_city")]:
    mask = contar_nomes_invalidos(df, coluna)
    print(f"{nome_df}.{coluna}: {mask.sum()} valores com caracteres fora do
esperado (de {len(df)})")

mask_sigla = geo["geolocation_city"].str.lower().isin({s.lower() for s in
estados_validos})
print("\ngeo.geolocation_city igual a uma sigla de estado (inválido):",
mask_sigla.sum())
print(geo.loc[mask_sigla, "geolocation_city"].value_counts())
```

```
customers.customer_city: 0 valores com caracteres fora do esperado (de 99441)
sellers.seller_city: 7 valores com caracteres fora do esperado (de 3095)
geo.geolocation_city: 12 valores com caracteres fora do esperado (de 1000163)

geo.geolocation_city igual a uma sigla de estado (inválido): 4
geolocation_city
sp    3
rj    1
Name: count, dtype: int64
```

**Achado 7 - categorias/texto inconsistente:**

- Estados ( `customer_state` , `seller_state` , `geolocation_state` ) estão todos dentro das 27 UFs válidas - sem problema aqui.
- Duas categorias de produto ( `pc_gamer` e `portateis_cozinha_e_preparadores_de_alimentos` ) não têm entrada na tabela

de tradução - ficarão sem `product_category_name_english` após a junção; é uma lacuna pequena e será documentada, não inventada.

- `geolocation_city` tem qualidade bem inferior às colunas equivalentes de `customers` / `sellers`: ~16 linhas com texto corrompido (entidades HTML como `florianópolis`, trechos de URL-encoding, pontuação solta) e 4 linhas em que o valor da cidade é na verdade uma sigla de estado (`sp`, `rj`) - tudo isso é inválido como nome de cidade. Além disso, a mesma cidade aparece sob múltiplas grafias (ex.: 22 variantes diferentes contendo "paulo"), a maioria apenas por existirem *outras* cidades cujo nome contém "paulo" (ex. "paulo afonso"), não por erro - mas há também duplicação de grafia genuína (com/sem acento). Corrigir a canonicalização completa de ~8.000 nomes de cidade exigiria uma tabela de referência externa e está fora do escopo do RA1; a decisão é nulificar apenas os ~20 valores claramente inválidos e documentar a limitação residual.

# Outliers (coordenadas fora do território brasileiro)

In [12]:
```
fora_do_brasil = geo[
    (geo["geolocation_lat"] > 6) | (geo["geolocation_lat"] < -34)
    | (geo["geolocation_lng"] > -32) | (geo["geolocation_lng"] < -74)
]
print(f"{len(fora_do_brasil)} de {len(geo)} coordenadas fora da caixa delimitadora aproximada do Brasil")
fora_do_brasil.head()
```

```
31 de 1000163 coordenadas fora da caixa delimitadora aproximada do Brasil
```

Out[12]:

| | geolocation_zip_code_prefix | geolocation_lat | geolocation_lng | geolocation_city | geolocation_state |
|---|---|---|---|---|---|
| **387565** | 18243 | 28.008978 | -15.536867 | bom retiro da esperanca | SP |
| **513631** | 28165 | 41.614052 | -8.411675 | vila nova de campos | RJ |
| **513643** | 28155 | -34.586422 | -58.732101 | santa maria | RJ |
| **513754** | 28155 | 42.439286 | 13.820214 | santa maria | RJ |
| **514429** | 28333 | 38.381672 | -6.328200 | raposo | RJ |

**Achado 8 - outliers de geolocalização:** 31 registros têm latitude/longitude fora da caixa delimitadora aproximada do território brasileiro (lat entre -34 e 6, lng entre -74 e -32) - coordenadas de geocodificação claramente erradas para um dataset que é 100% de pedidos no Brasil. Serão removidos na limpeza, pois usá-los em qualquer cálculo de distância nas próximas etapas produziria resultados sem sentido.

# Formato dos identificadores de CEP (zip code prefix)

In [13]:
```
print("prefixos de CEP < 10000 (perderiam o zero à esquerda se exibidos como string sem padding):")
print("customers:", (customers["customer_zip_code_prefix"] < 10000).sum(), "de", len(customers))
print("sellers:", (sellers["seller_zip_code_prefix"] < 10000).sum(), "de", len(sellers))
print("geo:", (geo["geolocation_zip_code_prefix"] < 10000).sum(), "de", len(geo))
```

```
prefixos de CEP < 10000 (perderiam o zero à esquerda se exibidos como string sem
padding):
customers: 23995 de 99441
sellers: 1027 de 3095
geo: 245733 de 1000163
```

**Achado 9 - tipo de dado incorreto (semântico):** os prefixos de CEP estão como número inteiro, mas CEP é um identificador categórico, não uma quantidade - e boa parte dos prefixos reais começam com `0` (região de São Paulo), que se perde na representação numérica ao exibir/exportar. Serão convertidos para texto com 5 dígitos (zero-padded) na limpeza.

# Resumo dos achados

| Tabela | Problema | Ação planejada |
|---|---|---|
| orders | datas como texto | converter para datetime |
| orders | nulos em datas de aprovação/entrega | manter (ausência legítima por status) |
| items | `shipping_limit_date` como texto | converter para datetime |
| payments | 3 linhas `not_defined` + valor 0 | remover |
| payments | 2 linhas com `installments == 0` | corrigir para 1 |
| reviews | datas como texto | converter para datetime |
| reviews | `review_id` duplicado entre pedidos | manter, documentar (não é erro) |
| products | 610 linhas sem categoria/metadados | manter nulo (ausência legítima) |
| products | 4 linhas com peso 0 | converter para nulo (valor impossível) |
| products | 2 categorias sem tradução em inglês | manter nulo após junção, documentar |
| sellers/customers | prefixo de CEP numérico | converter para texto com 5 dígitos |
| geo | ~26% duplicatas de linha inteira | remover |
| geo | 31 coordenadas fora do Brasil | remover |
| geo | ~20 nomes de cidade inválidos/corrompidos | converter para nulo |
| geo | prefixo de CEP numérico | converter para texto com 5 dígitos |

Detalhamento completo em `docs/03_relatorio_qualidade.md`. Próxima etapa: `03_limpeza_preparacao.ipynb`.

**Resposta do grupo:**

Avaliação completa nas células acima e em `docs/03_relatorio_qualidade.md`. Principais achados:

- **Tipos:** datas chegam como texto; prefixos de CEP chegam como inteiro, embora sejam identificadores (vários começam com `0`).
- **Ausentes (todos legítimos):** datas de aprovação/entrega nulas em pedidos que não chegaram a essa etapa; comentários de review opcionais; 610 produtos sem categoria.
- **Duplicados:** 261.831 linhas duplicadas (~26%) em `olist_geolocation_dataset.csv`; as outras 8 tabelas não têm.
- **Inválidos:** 3 pagamentos `not_defined` com valor 0, 2 com 0 parcelas e 4 produtos com peso 0.
- **Categorias e outliers:** ~20 valores corrompidos em `geolocation_city`, 2 categorias sem tradução e 31 pontos de geolocalização fora do Brasil; nenhum outlier de preço/frete/pagamento a tratar.

## 4. Limpeza e preparação dos dados

Realize a limpeza necessária ao projeto. Verifique valores ausentes, duplicados e inconsistências e trate os problemas encontrados, quando necessário.

# 03 - Limpeza e Preparação dos Dados

Trata, tabela por tabela, apenas os problemas identificados em `02_avaliacao_qualidade.ipynb` - sem aplicar técnicas que os dados não exigem. Cada decisão é justificada na célula de markdown correspondente. O resultado é salvo em `data/processed/`, mantendo a mesma granularidade dos arquivos originais (um CSV por tabela): a etapa de **juntar** as tabelas para análise é transformação e fica para as próximas etapas do projeto.

In [1]:
```
import re
from pathlib import Path

import numpy as np
import pandas as pd

pd.set_option("display.max_columns", None)

RAW = Path("../data/raw")
PROCESSED = Path("../data/processed")
PROCESSED.mkdir(parents=True, exist_ok=True)

orders = pd.read_csv(RAW / "olist_orders_dataset.csv")
customers = pd.read_csv(RAW / "olist_customers_dataset.csv")
items = pd.read_csv(RAW / "olist_order_items_dataset.csv")
payments = pd.read_csv(RAW / "olist_order_payments_dataset.csv")
reviews = pd.read_csv(RAW / "olist_order_reviews_dataset.csv")
products = pd.read_csv(RAW / "olist_products_dataset.csv")
sellers = pd.read_csv(RAW / "olist_sellers_dataset.csv")
geo = pd.read_csv(RAW / "olist_geolocation_dataset.csv")
cat_translation = pd.read_csv(RAW / "product_category_name_translation.csv")
```

## orders

**Decisão:** converter as 5 colunas de data/hora de texto para `datetime`. Os nulos em `order_approved_at`, `order_delivered_carrier_date` e `order_delivered_customer_date` **não são imputados**: eles ocorrem porque o pedido nunca passou por aquela etapa (cancelado, indisponível, ainda em processamento/transporte) - preencher esse valor inventaria um evento que não aconteceu. Nenhuma linha duplicada a remover (confirmado na avaliação).

In [2]:
```
colunas_data_orders = [
    "order_purchase_timestamp", "order_approved_at",
"order_delivered_carrier_date",
    "order_delivered_customer_date", "order_estimated_delivery_date",
]
for col in colunas_data_orders:
    orders[col] = pd.to_datetime(orders[col])

orders.dtypes
```

Out[2]:
```
order_id                              str
customer_id                           str
order_status                          str
order_purchase_timestamp    datetime64[us]
order_approved_at           datetime64[us]
order_delivered_carrier_date datetime64[us]
order_delivered_customer_date datetime64[us]
order_estimated_delivery_date datetime64[us]
dtype: object
```

# items

**Decisão:** converter `shipping_limit_date` para `datetime`. `price` e `freight_value` já são numéricos e sem valores inválidos (todos > 0 e >= 0, confirmado na avaliação) - nenhuma correção necessária.

In [3]:
```
items["shipping_limit_date"] = pd.to_datetime(items["shipping_limit_date"])
items.dtypes
```

Out[3]:
```
order_id                       str
order_item_id                int64
product_id                     str
seller_id                      str
shipping_limit_date    datetime64[us]
price                         float64
freight_value                 float64
dtype: object
```

# payments

**Decisões:**

1. Remover as 3 linhas com `payment_type == "not_defined"` e `payment_value == 0`: não representam uma forma de pagamento real nem um valor cobrado, e não há como inferir o tipo correto - mantê-las introduziria uma categoria inválida em qualquer contagem por `payment_type`.

2. Corrigir `payment_installments == 0` para `1` nas 2 linhas afetadas: zero parcelas é fisicamente impossível quando há um valor de pagamento registrado (mesmo um pagamento único conta como 1 parcela neste esquema).
3. **Não remover** as 6 linhas restantes com `payment_value == 0` do tipo `voucher`: são parcelas de pagamentos fracionados em múltiplos vouchers (`payment_sequential` > 1) e são legítimas.

In [4]:
```
linhas_antes = len(payments)

payments = payments[~((payments["payment_type"] == "not_defined") &
(payments["payment_value"] == 0))].copy()
print(f"Removidas {linhas_antes - len(payments)} linhas (not_defined + valor 0)")

payments.loc[payments["payment_installments"] == 0, "payment_installments"] = 1

print("payment_installments == 0 após correção:",
(payments["payment_installments"] == 0).sum())
print("payment_type após limpeza:", payments["payment_type"].unique())
```

```
Removidas 3 linhas (not_defined + valor 0)
payment_installments == 0 após correção: 0
payment_type após limpeza: <StringArray>
['credit_card', 'boleto', 'voucher', 'debit_card']
Length: 4, dtype: str
```

# reviews

**Decisões:**

1. Converter `review_creation_date` e `review_answer_timestamp` para `datetime`.
2. **Não** remover/imputar os nulos em `review_comment_title` / `review_comment_message`: são campos opcionais - ausência de texto é uma resposta válida ("não comentou"), não um dado faltante por erro.
3. **Não** tratar os 814 `review_id` duplicados como duplicatas a remover: o par `(review_id, order_id)` é único, ou seja, cada linha é uma avaliação legítima de um pedido específico - é só o identificador de review que é reaproveitado entre pedidos diferentes, uma particularidade conhecida deste dataset. Documentamos que `review_id` não deve ser tratado como chave primária em etapas futuras.

In [5]:
```
reviews["review_creation_date"] = pd.to_datetime(reviews["review_creation_date"])
reviews["review_answer_timestamp"] =
pd.to_datetime(reviews["review_answer_timestamp"])
reviews.dtypes
```

Out[5]:
```
review_id                      str
order_id                       str
review_score                 int64
review_comment_title           str
review_comment_message         str
review_creation_date    datetime64[us]
review_answer_timestamp datetime64[us]
dtype: object
```

# products

**Decisões:**

1. `product_weight_g == 0` (4 linhas) → converter para `NaN`: peso zero é fisicamente impossível para um produto físico; melhor marcar como ausente do que manter um valor sabidamente errado.
2. Nulos em `product_category_name` e metadados associados (~610 linhas) e nas dimensões físicas de 2 produtos → **mantidos como `NaN`**: são produtos cadastrados de forma incompleta pelo vendedor; inventar categoria ou dimensão seria fabricar dado.
3. Padronizar `product_category_name` ( `strip` ) e acrescentar `product_category_name_english` via junção com `product_category_name_translation` - isso resolve a inconsistência de a categoria estar só em português. As 2 categorias sem tradução ( `pc_gamer` , `portateis_cozinha_e_preparadores_de_alimentos` ) ficam com `product_category_name_english` nulo; é uma lacuna da tabela de tradução original, documentada, não inventada.

In [6]:
```
products["product_category_name"] = products["product_category_name"].str.strip()

products.loc[products["product_weight_g"] == 0, "product_weight_g"] = np.nan

cat_translation["product_category_name"] =
cat_translation["product_category_name"].str.strip()
products = products.merge(cat_translation, on="product_category_name", how="left")

print("product_weight_g == 0 após correção:", (products["product_weight_g"] ==
0).sum())
print("categorias sem tradução após junção:",
      products.loc[products["product_category_name"].notna(),
"product_category_name_english"].isna().sum())
products.head(3)
```

```
product_weight_g == 0 após correção: 0
categorias sem tradução após junção: 13
```

Out[6]:

| | product_id | product_category_name | product_name_lenght | product_description_lenght | product_photos_qty | product_weight_g | product_length_cm | product_height_cm | product_width_cm | product_category_name_english |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1e9e8ef04dbcff4541ed26657ea517e5 | perfumaria | 40.0 | 287.0 | 1.0 | 225.0 | 16.0 | 10.0 | 14.0 | perfumery |
| 1 | 3aa071139cb16b67ca9e5dea641aaa2f | artes | 44.0 | 276.0 | 1.0 | 1000.0 | 30.0 | 18.0 | 20.0 | art |
| 2 | 96bd76ec8810374ed1b65e291975717f | esporte_lazer | 46.0 | 250.0 | 1.0 | 154.0 | 18.0 | 9.0 | 15.0 | sports_leisure |

# customers e sellers

**Decisão:** converter `customer_zip_code_prefix` / `seller_zip_code_prefix` de número inteiro para texto com 5 dígitos (zero-padded). CEP é um identificador categórico - tratá-lo como número faz o prefixo perder o zero à esquerda em qualquer exibição/exportação (ex.: região de São Paulo, que começa com `0` ), mesmo que o valor numérico interno esteja correto. Estados já são 100% válidos (confirmado na avaliação) - nenhuma correção.

In [7]:
```
customers["customer_zip_code_prefix"] =
customers["customer_zip_code_prefix"].astype(str).str.zfill(5)
sellers["seller_zip_code_prefix"] =
sellers["seller_zip_code_prefix"].astype(str).str.zfill(5)

customers[["customer_zip_code_prefix"]].head(3)
```

Out[7]:

| | customer_zip_code_prefix |
|---|---|
| 0 | 14409 |
| 1 | 09790 |
| 2 | 01151 |

# geolocation

**Decisões:**

1. Remover as ~261.831 linhas 100% duplicadas: não adicionam nenhuma informação nova e infladavam o arquivo em ~26%.
2. Remover as 31 linhas com latitude/longitude fora da caixa delimitadora aproximada do Brasil: são erros de geocodificação (outliers) que não podem representar um endereço real de um pedido brasileiro.
3. Converter `geolocation_city` claramente inválido (igual a uma sigla de estado, ou com entidades HTML / URL-encoding / pontuação solta - ~20 linhas no total) para `NaN`, em vez de manter texto corrompido.
4. **Não** normalizar as milhares de variações de grafia legítimas (com/sem acento, espaçamento) de nomes de cidade: exigiria uma tabela de referência externa de municípios, o que está fora do escopo do RA1. Isso é documentado como limitação conhecida - análises futuras que precisem de granularidade de cidade devem preferir `customer_city` / `seller_city` (já validados) ou o prefixo de CEP.
5. Converter `geolocation_zip_code_prefix` para texto com 5 dígitos, pelo mesmo motivo de `customers` / `sellers`.

In [8]:
```
linhas_antes = len(geo)
geo = geo.drop_duplicates().copy()
print(f"Removidas {linhas_antes - len(geo)} linhas duplicadas")

fora_do_brasil = (
    (geo["geolocation_lat"] > 6) | (geo["geolocation_lat"] < -34)
    | (geo["geolocation_lng"] > -32) | (geo["geolocation_lng"] < -74)
)
print(f"Removendo {fora_do_brasil.sum()} coordenadas fora do Brasil")
geo = geo[~fora_do_brasil].copy()
```

```
Removidas 261831 linhas duplicadas
Removendo 27 coordenadas fora do Brasil
```

In [9]:
```
estados_validos = {

"AC","AL","AM","AP","BA","CE","DF","ES","GO","MA","MG","MS","MT","PA","PB","PE","P
I",
    "PR","RJ","RN","RO","RR","RS","SC","SE","SP","TO",
}
padrao_valido = re.compile(r"^[a-z0-9áéíóúãõâêôçàü\s\-\.\(\)/']*$")

cidade_e_sigla = geo["geolocation_city"].str.lower().isin({s.lower() for s in
```

```
estados_validos})
cidade_com_caractere_invalido =
~geo["geolocation_city"].fillna("").str.match(padrao_valido)
cidade_invalida = cidade_e_sigla | cidade_com_caractere_invalido

print(f"Nulificando {cidade_invalida.sum()} valores de geolocation_city
inválidos")
geo.loc[cidade_invalida, "geolocation_city"] = np.nan

geo["geolocation_zip_code_prefix"] =
geo["geolocation_zip_code_prefix"].astype(str).str.zfill(5)

geo.shape
```

```
Nulificando 16 valores de geolocation_city inválidos
```

Out[9]: (738305, 5)

# Verificação final e exportação

In [10]:
```
tabelas_finais = {
    "olist_orders_dataset.csv": orders,
    "olist_customers_dataset.csv": customers,
    "olist_order_items_dataset.csv": items,
    "olist_order_payments_dataset.csv": payments,
    "olist_order_reviews_dataset.csv": reviews,
    "olist_products_dataset.csv": products,
    "olist_sellers_dataset.csv": sellers,
    "olist_geolocation_dataset.csv": geo,
    "product_category_name_translation.csv": cat_translation,
}

for nome, df in tabelas_finais.items():
    assert df.duplicated().sum() == 0, f"{nome} ainda tem duplicatas de linha
inteira"

print("Nenhuma duplicata de linha inteira restante em nenhuma tabela.")
assert (payments["payment_installments"] == 0).sum() == 0
assert (products["product_weight_g"] == 0).sum() == 0
assert geo["geolocation_lat"].between(-34, 6).all()
assert geo["geolocation_lng"].between(-74, -32).all()
print("Checagens de valor inválido/outlier passaram.")
```

```
Nenhuma duplicata de linha inteira restante em nenhuma tabela.
Checagens de valor inválido/outlier passaram.
```

In [11]:
```
for nome, df in tabelas_finais.items():
    df.to_csv(PROCESSED / nome, index=False)

resumo = pd.DataFrame(
    [(nome, df.shape[0], df.shape[1]) for nome, df in tabelas_finais.items()],
    columns=["arquivo", "linhas", "colunas"],
)
resumo
```

Out[11]:

| | arquivo | linhas | colunas |
|---|---|---|---|
| 0 | olist_orders_dataset.csv | 99441 | 8 |
| 1 | olist_customers_dataset.csv | 99441 | 5 |
| 2 | olist_order_items_dataset.csv | 112650 | 7 |
| 3 | olist_order_payments_dataset.csv | 103883 | 5 |
| 4 | olist_order_reviews_dataset.csv | 99224 | 7 |
| 5 | olist_products_dataset.csv | 32951 | 10 |
| 6 | olist_sellers_dataset.csv | 3095 | 4 |
| 7 | olist_geolocation_dataset.csv | 738305 | 5 |
| 8 | product_category_name_translation.csv | 71 | 2 |

# Dataset resultante do RA1

Os 9 arquivos tratados foram salvos em `data/processed/`, com os mesmos nomes e a mesma granularidade dos originais (a única junção é a tradução das categorias de produto, que é uma padronização; agregações e outras junções são transformação e ficam para as próximas etapas). As principais mudanças em relação a `data/raw/`:

- colunas de data/hora como `datetime` em vez de texto;
- 3 pagamentos inválidos (`not_defined` + valor 0) removidos; 2 corrigidos (`installments` 0 → 1);
- peso de produto 0 → `NaN` (4 linhas);
- categoria de produto traduzida para inglês via junção com a tabela de tradução;
- prefixos de CEP padronizados como texto de 5 dígitos em `customers`, `sellers` e `geolocation`;
- `geolocation` sem duplicatas (-26%), sem coordenadas fora do Brasil e sem nomes de cidade corrompidos.

Ver `docs/04_dicionario_dados.md` para o dicionário de dados completo de cada arquivo resultante, e `docs/03_relatorio_qualidade.md` para o relatório de qualidade detalhado.

**Resposta do grupo:**

Cada decisão está justificada nas células de Markdown acima e resumida em `docs/03_relatorio_qualidade.md`:

- **Tipos:** datas convertidas para `datetime`; CEP convertido para texto de 5 dígitos.
- **Removidos:** as 261.831 duplicatas da geolocalização, os 3 pagamentos `not_defined` e 27 pontos fora do Brasil (31 antes de remover as duplicatas).
- **Corrigidos:** 0 parcelas → 1; peso 0 → nulo; valores inválidos de `geolocation_city` → nulo.
- **Mantidos de propósito:** os nulos legítimos (imputar inventaria dado que não existe), o `review_id` repetido e a grafia das cidades em `geolocation_city` (padronizar exigiria uma base externa de municípios; fica como limitação documentada).

# 5. Transformação dos dados

Realize as transformações necessárias à análise, como criação de colunas, filtros, agrupamentos, conversões ou junções, quando aplicáveis.

In [1]:
```
from pathlib import Path

import pandas as pd

PROCESSED = Path("../data/processed")
resumo = pd.DataFrame(
    [(p.name, *pd.read_csv(p).shape) for p in sorted(PROCESSED.glob("*.csv"))],
    columns=["arquivo", "linhas", "colunas"],
)
resumo
```

Out[1]:

| | arquivo | linhas | colunas |
|---|---|---|---|
| 0 | olist_customers_dataset.csv | 99441 | 5 |
| 1 | olist_geolocation_dataset.csv | 738305 | 5 |
| 2 | olist_order_items_dataset.csv | 112650 | 7 |
| 3 | olist_order_payments_dataset.csv | 103883 | 5 |
| 4 | olist_order_reviews_dataset.csv | 99224 | 7 |
| 5 | olist_orders_dataset.csv | 99441 | 8 |
| 6 | olist_products_dataset.csv | 32951 | 10 |
| 7 | olist_sellers_dataset.csv | 3095 | 4 |
| 8 | product_category_name_translation.csv | 71 | 2 |

**Resposta do grupo:**

Nesta etapa **não foram feitas transformações analíticas**: sem agregações nem indicadores. Além das conversões de tipo (seção 4), houve apenas a remoção de linhas inválidas, a correção de valores impossíveis e **uma única junção**, que traz a categoria do produto em inglês para `products` (padronização de categorias, coluna `product_category_name_english`). O dataset preparado mantém a mesma granularidade dos arquivos originais, um CSV tratado por tabela em `data/processed/`, como mostra o resumo abaixo. As demais junções e o cálculo de indicadores ficam para as próximas etapas. As colunas `dias_atraso` e `atrasado` só aparecem no stream do RA2 (seção 9).

# 7. Considerações do RA1

Registre brevemente:

- O que o grupo descobriu sobre os dados.
- Quais dificuldades ou limitações encontrou.
- Como os dados preparados poderão ser utilizados nas próximas etapas.

**Resposta do grupo:**

- **Descobertas:** problemas reais de naturezas diferentes - ~26% de linhas duplicadas na geolocalização, cidades corrompidas, valores impossíveis (0 parcelas, peso 0) e muitos nulos que **não são erro**, e sim etapas que o pedido não cumpriu.
- **Limitações:** a grafia de `geolocation_city` tem milhares de variações (preferir `customer_city` / `seller_city` ou o prefixo de CEP); 2 categorias sem tradução; `review_id` não serve como chave primária.
- **Uso adiante:** os 9 CSVs de `data/processed/` (dicionário em `docs/04_dicionario_dados.md`) são a base para juntar as tabelas pelas chaves e responder às perguntas; o RA2 já os usa como referência para a simulação.

---

# RA2 - Streaming Data Processing and Time Windows

Utilize os dados e o contexto do projeto para demonstrar o processamento em streaming e a aplicação de janelas temporais. Escolha um dos cenários:

- **Cenário A - Dados estáticos:** simule a chegada de novos registros utilizando dados artificiais coerentes com o contexto do projeto.
- **Cenário B - Dados em streaming:** utilize sua própria fonte de dados contínuos, sem necessidade de simulação.

## 8. Objetivo da análise em streaming

Responda brevemente:

- O que o grupo deseja monitorar ou analisar?
- Qual informação espera obter com o processamento em streaming?

**Resposta do grupo:**

Queremos acompanhar a **taxa de atraso de entrega** (pedidos entregues depois da data estimada) a cada **10 minutos**, conforme novos pedidos chegam. Isso se liga às perguntas 1 e 2 do RA1, sobre atrasos de entrega.

## 9. Fonte de dados e processamento

- Informe o cenário escolhido: dados reais em streaming ou simulação.
- Explique brevemente como os novos registros chegam.
- Implemente o processamento em tempo real.
- Demonstre o recebimento de novos registros.

**Resposta do grupo:**

**Cenário A (simulação):** a base da Olist é fixa, então criamos **15 pedidos artificiais**, escritos à mão, no formato da tabela `orders` (pedido, estado, data da entrega e data estimada).

- **Chegada:** os pedidos são divididos em **3 lotes** e gravados em arquivos CSV na pasta `data/stream_input/`, um de cada vez, com uma pausa entre eles.
- **Processamento:** o Spark vigia a pasta e processa cada arquivo novo. Para cada pedido, calcula `dias_atraso` (entrega menos estimada) e `atrasado` (1 se `dias_atraso` for maior que 0).
- **Ambiente:** rodamos em um container Linux (`docker/Dockerfile`), porque o Spark no Windows exige o `winutils`.

In [1]:
```
import shutil
import time
```

```
import warnings
from pathlib import Path

import pandas as pd
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

warnings.filterwarnings("ignore")  # esconde um aviso do PySpark sobre a versao do pandas

spark = (
    SparkSession.builder.master("local[2]")
    .appName("ra2-streaming")
    .config("spark.sql.shuffle.partitions", "2")
    .config("spark.ui.enabled", "false")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR")
print("Spark", spark.version)
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/21 00:41:26 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark 4.2.0
```

**Pedidos simulados:** 3 lotes de 5 pedidos, entregues no dia 01/05/2018 entre 12:00 e 12:35.

In [2]:
```
colunas = ["order_id", "customer_state", "entrega", "estimada"]

# Cada pedido: (id, estado, data e hora da entrega, data estimada de entrega)
lote1 = [
    ("p01", "SP", "2018-05-01 12:01:10", "2018-05-10"),
    ("p02", "RJ", "2018-05-01 12:03:45", "2018-04-29"),
    ("p03", "MG", "2018-05-01 12:05:20", "2018-05-08"),
    ("p04", "SP", "2018-05-01 12:07:05", "2018-05-12"),
    ("p05", "BA", "2018-05-01 12:08:30", "2018-04-30"),
]
lote2 = [
    ("p06", "SP", "2018-05-01 12:11:15", "2018-05-09"),
    ("p07", "RS", "2018-05-01 12:14:40", "2018-04-27"),
    ("p08", "PR", "2018-05-01 12:17:10", "2018-05-06"),
    ("p09", "SP", "2018-05-01 12:21:25", "2018-05-03"),
    ("p10", "MG", "2018-05-01 12:23:50", "2018-04-30"),
]
# o pedido p15 foi entregue as 12:12, mas só chega no lote 3 (chegada fora de ordem)
lote3 = [
    ("p11", "SC", "2018-05-01 12:26:30", "2018-05-05"),
    ("p12", "SP", "2018-05-01 12:28:10", "2018-04-29"),
    ("p13", "RJ", "2018-05-01 12:31:45", "2018-05-11"),
    ("p14", "SP", "2018-05-01 12:33:20", "2018-05-02"),
    ("p15", "BA", "2018-05-01 12:12:30", "2018-04-28"),
]

pd.DataFrame(lote1, columns=colunas)
```

Out[2]:

| | order_id | customer_state | entrega | estimada |
|---|---|---|---|---|
| **0** | p01 | SP | 2018-05-01 12:01:10 | 2018-05-10 |
| **1** | p02 | RJ | 2018-05-01 12:03:45 | 2018-04-29 |
| **2** | p03 | MG | 2018-05-01 12:05:20 | 2018-05-08 |
| **3** | p04 | SP | 2018-05-01 12:07:05 | 2018-05-12 |
| **4** | p05 | BA | 2018-05-01 12:08:30 | 2018-04-30 |

**Definição do stream:** o Spark lê a pasta de entrada e calcula, para cada pedido, `dias_atraso` e `atrasado`.

In [3]:
```
pasta = Path("../data/stream_input")
if pasta.exists():
    shutil.rmtree(pasta)
pasta.mkdir(parents=True)

esquema = "order_id STRING, customer_state STRING, entrega TIMESTAMP, estimada
DATE"

entrada = spark.readStream.schema(esquema).option("header", True).csv(str(pasta))

pedidos = (
    entrada
    .withColumn("dias_atraso", F.datediff(F.to_date("entrega"),
F.col("estimada")))
    .withColumn("atrasado", F.when(F.col("dias_atraso") > 0, 1).otherwise(0))
)
```

# 10. Aplicação de janelas temporais

- Utilize uma janela **Tumbling** (intervalos sem sobreposição) ou **Sliding** (intervalos que podem se sobrepor).
- Informe o tipo e a duração da janela e o deslocamento, quando aplicável.
- Implemente uma agregação coerente com a análise, como contagem, soma ou média.
- Explique brevemente o que será calculado em cada intervalo.

O foco é demonstrar a compreensão do conceito; não é necessário justificar tecnicamente a duração escolhida.

**Resposta do grupo:**

| Item | Valor |
|---|---|
| Tipo | **Tumbling** (janelas fixas, sem sobreposição) |
| Duração | **10 minutos** |
| Deslocamento | Não se aplica (em tumbling é igual à duração) |
| Data e hora usadas | `entrega` (momento da entrega) |
| Agregações | `count` (pedidos), `sum` (atrasados) e `avg` (taxa de atraso, em %) |
| Saída | `complete` : a tabela inteira é mostrada a cada lote |

**Em cada intervalo** calculamos: quantos pedidos foram entregues, quantos atrasaram e a taxa de atraso (%).

In [4]:
```
janelas = (
    pedidos
    .groupBy(F.window("entrega", "10 minutes"))
    .agg(
        F.count("*").alias("pedidos"),
        F.sum("atrasado").alias("atrasados"),
        F.round(F.avg("atrasado") * 100, 1).alias("taxa_atraso_pct"),
    )
)

consulta = (
    janelas.writeStream
    .outputMode("complete")
    .format("memory")
    .queryName("janelas")
    .start()
)
print("stream ativo:", consulta.isActive)
```

```
stream ativo: True
```

## 11. Resultados e interpretação

Apresente os resultados do processamento e demonstre como a análise se comporta com a chegada de novos registros.

**Chegada dos lotes:** cada chamada grava um arquivo novo na pasta e mostra a tabela das janelas.

In [5]:
```
def enviar_lote(numero, registros):
    """Grava um arquivo novo na pasta e espera o Spark processar."""
    pd.DataFrame(registros, columns=colunas).to_csv(pasta / f"lote_{numero}.csv",
index=False)
    consulta.processAllAvailable()
    print(f"Lote {numero} chegou ({len(registros)} pedidos). Janelas agora:")
    spark.sql("""
        SELECT date_format(window.start, 'HH:mm') AS inicio,
               date_format(window.end, 'HH:mm') AS fim,
               pedidos, atrasados, taxa_atraso_pct
        FROM janelas
        ORDER BY window.start
    """).show()


enviar_lote(1, lote1)
```

```
Lote 1 chegou (5 pedidos). Janelas agora:
+------+-----+-------+---------+---------------+
|inicio|  fim|pedidos|atrasados|taxa_atraso_pct|
+------+-----+-------+---------+---------------+
| 12:00|12:10|      5|        2|           40.0|
+------+-----+-------+---------+---------------+
```

In [6]:
```
time.sleep(2)  # intervalo entre as chegadas
enviar_lote(2, lote2)
```

```
Lote 2 chegou (5 pedidos). Janelas agora:
+------+-----+-------+---------+---------------+
|inicio|  fim|pedidos|atrasados|taxa_atraso_pct|
+------+-----+-------+---------+---------------+
| 12:00|12:10|      5|        2|           40.0|
| 12:10|12:20|      3|        1|           33.3|
| 12:20|12:30|      2|        1|           50.0|
+------+-----+-------+---------+---------------+
```

In [7]:
```
time.sleep(2)
enviar_lote(3, lote3)
```

```
Lote 3 chegou (5 pedidos). Janelas agora:
+------+-----+-------+---------+---------------+
|inicio|  fim|pedidos|atrasados|taxa_atraso_pct|
+------+-----+-------+---------+---------------+
| 12:00|12:10|      5|        2|           40.0|
| 12:10|12:20|      4|        2|           50.0|
| 12:20|12:30|      4|        2|           50.0|
| 12:30|12:40|      2|        0|            0.0|
+------+-----+-------+---------+---------------+
```

**Conferência:** a soma dos pedidos nas janelas deve ser igual ao total de pedidos enviados.

In [8]:
```
total = spark.sql("SELECT SUM(pedidos) FROM janelas").collect()[0][0]
print("Total de pedidos nas janelas:", total, "(de", len(lote1) + len(lote2) +
len(lote3), "enviados)")

consulta.stop()
spark.stop()
shutil.rmtree(pasta, ignore_errors=True)
```

```
Total de pedidos nas janelas: 15 (de 15 enviados)
```

**Resposta do grupo:**

- **Lote 1:** cria a janela `12:00-12:10` com 5 pedidos, 2 atrasados (40,0%).
- **Lote 2:** cria `12:10-12:20` (3 pedidos, 1 atrasado, 33,3%) e `12:20-12:30` (2 pedidos, 1 atrasado, 50,0%). A janela `12:00-12:10` não muda.
- **Lote 3:** cria `12:30-12:40` (2 pedidos, nenhum atrasado) e atualiza `12:20-12:30`, que passa a ter 4 pedidos, 2 atrasados (50,0%). O pedido das 12:12 só chega neste lote e atualiza a janela `12:10-12:20`, que passa a ter 4 pedidos, 2 atrasados (50,0%).
- A soma das janelas dá 15 pedidos, igual ao total enviado.

Os dados são simulados, então as taxas não representam a Olist. O que a demonstração mostra é o comportamento do processamento.

## 12. Considerações do RA2

Responda brevemente:

- O que foi possível observar com o processamento em streaming?
- Qual foi a contribuição das janelas temporais para a análise?
- Essa abordagem faria sentido em uma aplicação real do projeto? Por quê?

Na última resposta, o grupo pode reconhecer que o streaming foi utilizado apenas como exercício didático, se esse for o caso.

**Resposta do grupo:**

- **Observado:** a taxa de atraso por janela muda a cada chegada. Janelas novas aparecem, e janelas antigas são atualizadas quando chega um pedido que pertence a elas.
- **Contribuição das janelas:** dividem o fluxo contínuo em intervalos de 10 minutos que podem ser comparados.
- **Faria sentido na prática?** Em parte. Com entregas registradas continuamente, acompanhar o atraso por janela ajudaria a reagir a problemas logísticos. Aqui o streaming foi um **exercício didático** com dados simulados. Os dados da Olist são um histórico fechado (cerca de 130 pedidos por dia, em média), e janelas de horas ou de um dia seriam mais adequadas.